FaiRTT: An Empirical Approach for Enhanced RTT Fairness and Bottleneck Throughput in BBR

Akshita Abrol*, Purnima Murali Mohan*, Tram Truong-Huu*†
*Singapore Institute of Technology (SIT), Singapore
†Agency for Science, Technology and Research (A*STAR), Singapore
Email: {akshita.abrol, purnima.mohan, truonghuu.tram}@singaporetech.edu.sg

2024 IEEE International Conference on Communications Workshops (ICC Workshops) | 979-8-3503-0405-3/24/$31.00 ©2024 IEEE | DOI: 10.1109/ICCWORKSHOPS59551.2024.10615947

***Abstract*—In next-generation networks, achieving Round-trip Time (RTT) fairness is essential for ensuring fair bandwidth distribution among diverse flow types, enhancing overall network utilization. The TCP congestion control algorithm — BBR, was proposed by Google to dynamically adjust sending rates in response to changing network conditions. While BBRv2 was implemented to overcome the unfairness limitation of BBRv1, it still faces intra-protocol fairness challenges in balancing the demands of high-bandwidth, long-RTT elephant flows and more frequent short-RTT mice flows. These issues lead to throughput imbalances and queue buildup, resulting in elephant flow dominance and mice flow starvation. In this paper, we first investigate the limitations of Google's BBR algorithm, specifically in the context of intra-protocol RTT fairness in beyond 5G (B5G) networks. While existing works address this limitation by adjusting the pacing rate, it eventually leads to low throughput. We hence develop the FaiRTT algorithm to resolve the problem by dynamically estimating the Bandwidth Delay Product (BDP) sending rate based on RTT measurements, focusing on equitable bandwidth allocation. By modeling the $Inflight$ dependency on the BDP, bottleneck bandwidth, and packet departure time after every `ACK`, we can resolve the intra-protocol fairness while not compromising the throughput on the bottleneck link. Through extensive simulations on NS-3 and comprehensive performance evaluations, FaiRTT is shown to significantly improve the fairness index and network throughput, significantly outperforming BBRv2, for diverse flow types. FaiRTT achieves an average throughput ratio of 1.08 between elephant and mice flows, an average fairness index of 0.98, and an average utilization of the bottleneck link of 98.78%.**

***Index Terms*—BBR, Traffic Congestion Control, Flow Fairness, Network Throughput, Elephant and Mice Flows.**

I. Introduction

The Bottleneck Bandwidth and Round-trip propagation time (BBR) algorithm, introduced by Google in 2016 [1], stands out as a pivotal mechanism, especially considering its capability to dynamically adjust sending rates in response to real-time network conditions. The versatility of BBR makes it an indispensable tool, not merely for enhancing throughput and reducing latency in the ever-evolving and complex network landscapes but also for effectively navigating the challenges brought forth by the continuous advancement in network technologies and their accompanying protocols. In next-generation networks (NGN) such as beyond 5G (B5G) or 6G, the analysis of flow size statistics showcases a consistent pattern of *heavy-tail* behavior for applications ranging from bandwidth-demanding long flows (e.g., video-surveillance drone applications, AR/VR) to the delay-sensitive short flows [2]. Such heavy-tail flow distribution in NGN is similar to that of the Internet's familiar *mice-elephant* phenomenon.

The high-bandwidth, high Round-Trip Time (RTT) applications, which account for a smaller fraction of the overall network traffic, are referred to as the "elephant" flows. This contrasts sharply with the plethora of smaller "mice" flows, such as tactile internet signaling traffic, HTTP requests, etc., which, despite their higher frequency, low-RTT constitute a lesser segment of the total traffic volume. In the B5G network landscape, the distinct roles of high-bandwidth, long-RTT elephant flows, and frequent short-RTT mice flows are even more pronounced with their varying QoS and RTT requirements. To model the network flows, BBR works on Kleinrock's optimal operating point [3] by estimating the Maximum Delivery Rate referred to as the Bottleneck Bandwidth (`BtlBw`) and the Minimum Round-Trip Time known as `RTprop`. Based on the Bandwidth Delay Product (BDP), measured as the product of `BtlBw` and `RTprop`, the algorithm calculates three control parameters: (i) sending quantum, (ii) pacing rate, and (iii) congestion window. BBR estimates bottleneck bandwidth to set sending rates — aiming for high throughput without increasing the waiting time of the packets in the queue [4].

However, the first version of BBR (BBRv1) overestimates the sending rate by constantly filling the BDP leading to queue buildup, bandwidth, and RTT unfairness issues [5]–[7]. BBRv2 was developed to overcome these limitations by introducing mechanisms to better estimate and adapt to the actual available bandwidth and minimize queue buildup. While BBRv2 adapts better to network changes than BBRv1, it still struggles in NGN networks with highly varying QoS requirements. Ensuring complete fairness among multiple BBRv2 flows is challenging, especially in NGN scenarios of the coexistence of elephant and mice flows due to the overestimation (elephant flows) or underestimation (mice flows) of the optimal operating point. For example, the elephant flows tend to dominate, causing unfairness and queue build-up at the router while the mice flows face starvation due to the unintended shift in the operating point due to the overestimation. Consequently, this unintended shift in the operating point leads to slow, unstable convergence, RTT unfairness, and throughput underutilization by different flow RTTs [8], [9]. Some efforts have been made to improve the intra-protocol fairness for BBRv1 in [8] and BBRv2 in [10]. They utilize queue size estimation to adjust the pacing gain to improve fairness. However, there is no emphasis on changing the BDP itself depending on the incoming flow.

In this paper, we develop a novel algorithm (named FaiRTT) that can dynamically estimate the BDP based on flow-centric RTT estimate to mitigate the observed fairness challenges.

Authorized licensed use limited to: NATIONAL INSTITUTE OF TECHNOLOGY SURATHKAL. Downloaded on October 06,2026 at 11:13:40 UTC from IEEE Xplore. Restrictions apply.

The primary objective is to ensure equal opportunity for both elephant and mice flows within the bottleneck link, addressing the dominance and starvation issues, respectively, prevalent in existing BBR implementations. We use the standardized Jain's fairness index as one of the key performance metrics to demonstrate the effectiveness of FaiRTT, combined with other network performance metrics such as throughput, and link utilization. Through the performance evaluation on NS-3, we show that FaiRTT has a better average throughput ratio of 1.08 between elephant and mice flows and an average fairness index of up to 0.98 compared to that of BBRv2 while improving the overall bottleneck link bandwidth utilization to 98.78%.

The rest of the paper is organized as follows. In Section II, we discuss the related work. We present the FaiRTT algorithm for improving RTT fairness in Section III. Section IV presents our experiments and analysis of results before we conclude the paper in Section V.

II. Related Work

Since the introduction of Google's BBR congestion control algorithm, numerous experiments and analyses have been performed to evaluate its performance in various network scenarios. In this section, we review several works and discuss potential issues that can happen to traffic flows.

A. BBR Operational Intricacies

In contrast to the initial promising results reported by Google on the effectiveness of BBRv1 (BBR-version 1), subsequent studies uncovered several issues in its behavior. Scholz *et al.* [5] identified the creation of long-standing queues during the startup phase, suppressing existing flows. Hock *et al.* [6] observed bandwidth discrepancies between elephant and mice flows sharing a bottleneck link due to the higher BDP of elephant flows. Scherrer *et al.* [7] found that in the presence of multiple flows managed by BBRv1, overestimated delivery rates lead to standing queues exceeding 1.5 times BDP, causing packet loss and unfair bandwidth shared among flows, particularly for small buffer sizes. To address the above concerns, Google introduced BBRv2 (BBR-version 2) [11] in 2018, incorporating explicit congestion notification (ECN) and packet loss rate. Zhang *et al.* [12] while evaluating various BBR versions, highlighted BBRv2's improvements in flow fairness and enhanced coexistence with CUBIC and Reno. However, suboptimal channel utilization was noted with a 5% loss rate. Song *et al.* [13] reported fairness enhancements and reduced re-transmissions with BBRv2, emphasizing its effectiveness in limited buffer scenarios but noting challenges in fair bandwidth convergence with two flows entering a bottleneck link at different times. Nandagiri *et al.* [14] conducted an experimental evaluation comparing BBRv1 and BBRv2, revealing BBRv2's ability to overcome inflight capacity limitations in networks with small buffers. However, in networks with large buffers, long RTT flows still consume more bandwidth.

B. BBR Enhancing Fairness

Ma *et al.* [15] introduced the BBQ algorithm to enhance RTT fairness by minimizing probing periods for flows with extended round-trip times. While effective, BBQ's performance degrades with an increasing number of competing flows. Kim *et al.* proposed solutions in [16] and [17] to limit BBRv1's inflight capacity during the ProbeBW phase, addressing bias towards long RTT flows and preventing excessive data transmission. However, it causes a decrement in total throughput due to limiting inflight capacity to 1 BDP to prevent congestion. In [18], the authors presented the Delay-Aware BBR (DA-BBR) algorithm to improve RTT fairness and throughput using the relation of each flow's RTT with `RTprop`. However, this algorithm was noted for slow and unstable convergence. Njogu *et al.* [8] introduced BBR-With Enhanced Fairness (BBR-EFRA), dynamically adjusting congestion window (CWND) based on buffer queue status for equitable competition among different RTT flows. Pan *et al.* [9] addressed the fairness and excessive re-transmissions in BBRv2 with the introduction of BBRv2+, utilizing flow-aware explicit congestion notification. However, these algorithms rely on accurate real-time queue estimation, which is challenging in dynamic network conditions.

Addressing these issues remains crucial for achieving equitable bandwidth allocation among different RTT flows without compromising on the throughput through a stable convergence. With the continuous updating of the BBR algorithm, BBRv2 can solve some fairness problems and limitations of BBRv1. BBRv3 (BBR-version 3) [19] has recently been released, aiming to rectify bugs and optimize performance parameters observed in BBRv2. While BBRv2 and BBRv3 present updates and improvements, experiments reveal persistent RTT unfairness. Thus, continued research is essential to enhance RTT fairness and intra-protocol fairness, providing potential refinements for the final version of BBR.

III. FaiRTT Algorithm

A. Overview of BBRv2

Given a network, the main objective of BBR is to maximize the utilization of the bottleneck link with a minimal delay. BBR estimates the bandwidth of the bottleneck link as the maximum observed delivery rate and the propagation delay as the minimum observed RTT over periodic intervals. The bandwidth of the bottleneck link is denoted as `BtlBw`, and the propagation delay is denoted as `RTprop`. However, sending more traffic (probing more bandwidth) to estimate `BtlBw` may cause congestion at the bottleneck link, thus increasing `RTprop` and vice-versa. BBR, therefore, estimates these values separately in its different execution phases.

In Fig. 1, we present the flow diagram of BBRv2. We provide the meaning of mathematical notations used in the BBRv2 algorithm in Table I. The BBRv2 algorithm is composed of four phases: Startup (`BBR_START_UP`), Drain (`BBR_DRAIN`), Probe Bandwidth (`BBR_PROBE_BW`), and Probe RTT (`BBR_PROBE_RTT`). The first phase (`BBR_START_UP`) adapts the exponential Startup behavior from CUBIC by doubling the sending rate with each RTT. The `pacing_gain` is set to 2.89. If the increase in the sending rate does not exceed 25% for three consecutive attempts or if packet loss or an ECN-marked rate surpasses a predefined threshold (e.g., 2%), BBRv2 assumes to have reached the bottleneck bandwidth. Since this observation is delayed by one RTT, a queue was already created at the bottleneck. In the second phase

Authorized licensed use limited to: NATIONAL INSTITUTE OF TECHNOLOGY SURATHKAL. Downloaded on October 06,2026 at 11:13:40 UTC from IEEE Xplore. Restrictions apply.

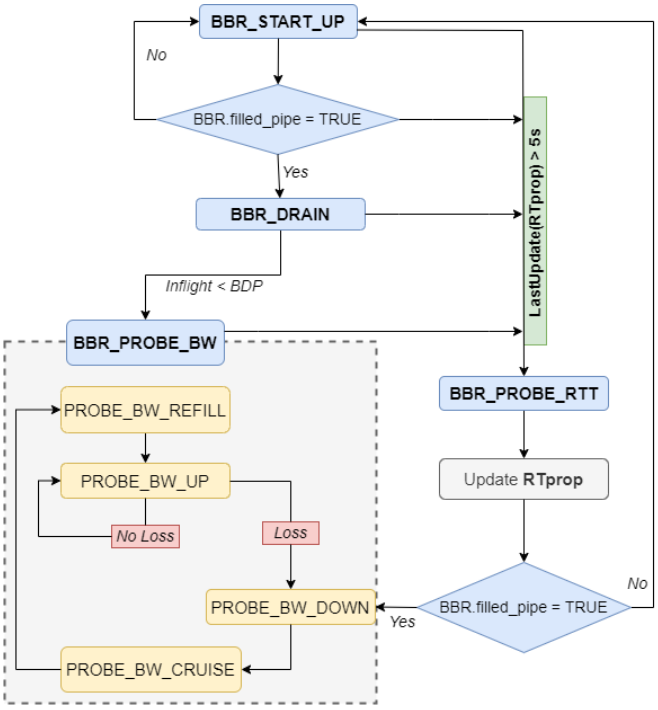


Fig. 1: BBRv2 Phases and Flow Diagram.

TABLE I: Mathematical Notations

Notation	Definition
BDP_t	Bandwidth delay product at time window t
$\texttt{RTprop}_t$	Minimum RTT captured at time window t
$\texttt{BtlBw}_t$	Max. bottleneck bandwidth at time window t
$lastRTT_t$	RTT of the last ACK at time window t
$minRTT_t$	Minimum value of RTT at time window t
$dRate_t$	Delivery rate of the bottleneck link at time window t
$maxBw_t$	Maximum delivery rate at time window t
$Inflight_t$	Estimated volume of in-flight data to utilize available bottleneck bandwidth at time window t
α_t	RTT fairness threshold at time window t
$Wfcount_t$	Number of flows at time window t
$WminRTT_t$	RTT estimate at time window t
β	Balance factor set to 0.8
γ	Discount factor set to 0.99
PROBE_INT_EXP	Expiration flag of the probe bandwidth phase
RATE_LIM_APP	Rate limiting flag

(BBR_DRAIN), BBRv2 tries to drain the queue by temporarily reducing the pacing_gain to approximately 0.34. Afterwards, BBRv2 enters the third phase (BBR_PROBE_BW), in which it probes for more available bandwidth. BBRv2 specifies exit conditions for each stage of the third phase (ProbeBW Refill, ProbeBW Up, ProbeBW Down, and ProbeBW Cruise). BBRv2 initiates bandwidth probing by linearly augmenting the amount of inflight data over one $RTprop$ in the ProbeBW Refill stage, effectively filling the network pipe. Subsequently, BBRv2 engages in the ProbeBW Up phase, during which it increases the inflight data swiftly to probe for more bandwidth. This increase of inflight data ceases when the volume reaches 1.25 times the BDP, or when packet loss or the ECN rate surpasses a predetermined threshold. BBRv2 establishes inflight_hi when confronted with a higher loss/ECN rate than the threshold, ensuring that it does not surpass the operational point where excessive packet loss may occur. Upon concluding the ProbeBW Up phase, BBRv2 transitions to the ProbeBW Down phase to offset the queue. It exits this phase once the volume of inflight data becomes less than BDP and maintains a constant delivery rate in ProbeBW Cruise until the initiation of the next bandwidth probing cycle. If BBRv2 experiences packet loss in the ProbeBW Cruise, it uses inflight_lo to cope with the temporary packet loss. BBRv2 continuously samples the bandwidth for BDP estimation using a time window basis. At the time window t, $\texttt{BtlBw}_t$ is defined as follows:

$$\texttt{BtlBw}_t = \begin{cases} maxBw_t, & dRate_t \geqslant \texttt{BtlBw}_{t-1}, \\ maxBw_t, & \texttt{RATE_LIM_APP} = \texttt{FALSE}, \\ \texttt{BtlBw}_{t-1}, & \text{otherwise} \end{cases} \tag{1}$$

where $maxBw_t$ is the maximum delivery rate captured from the socket within time window t, normally set to two times the probe bandwidth phase. It is to be noted that $\texttt{BtlBw}_t$ is measured only for the applications with no rate limit, indicated by the flag RATE_LIM_APP = FALSE.

After not measuring a new RTprop value for 5 seconds, BBRv2 stops probing for bandwidth and enters the last phase (Probe RTT). During this phase, the bandwidth is reduced to half of BDP to drain any possible queue and get a real estimation of the RTT. This phase is kept for 200 ms, controlled by the flag PROBE_INT_EXP. If a new minimum value is measured, $\texttt{RTprop}_t$ is updated as follows:

$$\texttt{RTprop}_t = \begin{cases} lastRTT_t, & 0 \leqslant lastRTT_t \leqslant \texttt{RTprop}_{t-1}, \\ lastRTT_t, & \texttt{PROBE_INT_EXP} = \texttt{TRUE}, \\ \texttt{RTprop}_{t-1}, & \text{otherwise} \end{cases} \tag{2}$$

where $lastRTT_t$ is captured from the network whenever an ACK packet is received.

The BBRv2 algorithm estimates the amount of data to be transmitted based on the current BDP. The BDP is in turn calculated by the product of the bottleneck bandwidth at time window t and the estimated RTT of the flows propagating in that time window, $\texttt{RTprop}_t$.

$$BDP_t = \texttt{BtlBw}_t \times \texttt{RTprop}_t. \tag{3}$$

B. Drawback of BBR

When multiple flows with diverse RTT coexist over a shared bottleneck link, BBR tends to transmit excess data, approximately 1.5 times the intended amount [5]. Consequently, BBR deviates from its typical operating point, leading to a total transmission rate exceeding the available bandwidth of the bottleneck link. This results in a persistent queue on the bottleneck link which is shared among all flows — predominantly favoring the elephant flows, causing bandwidth unfairness.

Elephant flows, with higher BDP values, transmit more packets and utilize more bottleneck buffers. Our experimental analysis reveals that the BDP value of mice flows becomes smaller than that of elephant flows. Consequently, mice flows inject fewer packets into the pipeline, reducing the delivery

Authorized licensed use limited to: NATIONAL INSTITUTE OF TECHNOLOGY SURATHKAL. Downloaded on October 06,2026 at 11:13:40 UTC from IEEE Xplore. Restrictions apply.

rate. The decrease in delivery rates amplifies the dominance of elephant flows, leading to increased throughput imbalance, higher packet loss, re-transmission rates, and elevated latency. In BBRv1, mice flows were found to be constrained by the congestion window, thereby reducing their transmission rates. BBRv2 aimed to mitigate this issue by using ECN and setting of `inflight` upper and lower bound. However, in [13], the authors reported that BBRv2 still experienced significant issues with coexistence between different RTT flows particularly with large buffers. Furthermore, in [20], the authors revealed that BBRv3 continues to exhibit RTT unfairness. Malicious actors can exploit this vulnerability, deliberately increasing delay to gain a disproportionately higher bandwidth share.

C. FaiRTT Algorithm

Algorithm 1 FaiRTT: RTT Fairness Improvement Algorithm

Input: $\texttt{BtlBw}_t$, $\texttt{RTprop}_t$, $cwnd_gain$, $lastRTT_t$
Input: $minRTT_t$, γ
Output: $Inflight_t$
$Wfcount_t \leftarrow \texttt{length}(\texttt{unique}(minRTT)$ in window $t)$
Calculate α_t using Eq. (5)
if BBR $\rightarrow$ mode == BBR_PROBE_BW **then**
  if $lastRTT_t > \alpha_t$ **then**
    $BDP_t \leftarrow \texttt{BtlBw}_t \times \texttt{RTprop}_t \times \left(\frac{minRTT_t \times \gamma}{lastRTT_t}\right)$
  else
    $BDP_t \leftarrow \texttt{BtlBw}_t \times \texttt{RTprop}_t$
  end if
else
  $BDP_t \leftarrow \texttt{BtlBw}_t \times \texttt{RTprop}_t$
end if
$Inflight_t \leftarrow BDP_t \times cwnd_gain$
return $Inflight_t$

To address the inherent design issue of BBR, we develop Algorithm 1, which dynamically captures the changes in RTT and adaptively adjusts inflight for each BBR flow request. The proposed algorithm aims to improve RTT fairness while maintaining the overall throughput of the network. We leverage the calculated RTT from each acknowledgment packet (`lastRTT`) and minimum RTT (`minRTT`) to calculate an adjustment coefficient for curtailing the amount of inflight data for each flow. The inflight is defined as follows:

$$Inflight_t = BDP_t \times cwnd_gain \tag{4}$$

where $cwnd_gain$ is fixed at 0.5 for the `BBR_PROBE_RTT` phase and at 2 for all other phases of BBRv2. While all BBR flows exceed their intended inflight data transmission, the magnitude differs, particularly with elephant flows transmitting significantly larger volumes. To account for this, we examine the relationship between `lastRTT` and `minRTT`, introducing an adjustment coefficient with a value below 1. This coefficient will approach 1 for mice flows, while for elephant flows it would have a smaller value. The BDP calculation incorporates the adjustment coefficient, dynamically adjusting inflight data based on RTT. The adjustment coefficient is multiplied with the BDP only when the value of `lastRTT` is more than the RTT fairness threshold α_t, calculated as:

$$\alpha_t = Wfcount_t \cdot \sum\nolimits_{j \in W_t} WminRTT_j / W_t \tag{5}$$

where

$$WminRTT_t = \beta \cdot \sum_{i=0}^{t-1} \frac{minRTT_i}{t} + (1-\beta) \cdot minRTT_t \tag{6}$$

and

$$W_t = \min(|\mathbb{R}\text{x}|, |\mathbb{T}\text{x}|) \tag{7}$$

where $|\mathbb{R}\text{x}|$ and $|\mathbb{T}\text{x}|$ are the size of the receiving window and sending window , respectively. $Wfcount_t$ is the estimated number of BBR flows in the time window t calculated by counting the number of unique $minRTT$'s in that time window, and β is a constant fixed at 0.8 to balance the estimate. Further, we introduce a discount factor γ of 0.99 to stabilize BDP and promote equitable bandwidth-sharing. Elephant flows exhibit a lower adjustment coefficient than mice flows, resulting in a more substantial reduction in elephant flow's BDP compared to that of mice flows. Consequently, by transmitting less data than the original BBR, elephant flows experience a significant reduction in buffer occupancy, thereby enhancing RTT fairness.

IV. Performance Evaluation

A. Experimental Setup

We used a dumbbell topology setup in the NS-3 simulation framework, featuring multiple flows sharing a common bottleneck link, as illustrated in Fig. 2. We applied the default active queue management as the Drop-Tail policy and every packet size as 1 kB. Each simulation lasted 120 seconds and was repeated 5 times with different random seeds for the error rate in packet delivery. The obtained results were plotted with 95% confidence interval.

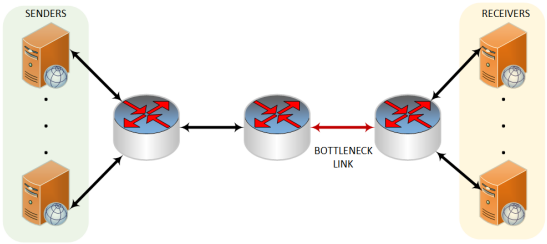


Fig. 2: Experimental network topology.

1) Performance Metrics: We used the following three performance metrics to evaluate the proposed algorithm.

- Throughput: We used average throughput to assess the equal distribution of bandwidth among competing flows. We defined the throughput ratio as the ratio of the throughput of elephant flows and that of mice flows.
- Fairness Index: We used Jain's fairness index [21] to provide a more comprehensive evaluation of fairness in the allocation of bandwidth among competing flows. The Jain's fairness index, F is calculated using the formula:

$$F = \left(\sum\nolimits_{i=1}^{n} T_i\right)^2 / \left(n \cdot \sum\nolimits_{i=1}^{n} T_i^2\right) \tag{8}$$

Here, n represents the number of competing flows, and T_i denotes the throughput of flow i. A value of F close to 1 signifies fairness in the allocation of bandwidth sharing among competing flows.

Authorized licensed use limited to: NATIONAL INSTITUTE OF TECHNOLOGY SURATHKAL. Downloaded on October 06,2026 at 11:13:40 UTC from IEEE Xplore. Restrictions apply.

- Bottleneck link utilization: Link utilization is vital for assessing network efficiency, measuring how effectively the bottleneck link handles data transmission. The percentage bottleneck link utilization U is computed as:

$$U(\%) = \frac{\sum_i \mathrm{T}_i}{C_{\mathrm{Btl}}} \cdot 100 \tag{9}$$

where T_i is the throughput of flow i traversing the bottleneck link and C_{Btl} is the bandwidth of the bottleneck link set to 10 Mbps for all experiments. Higher link utilization indicates better use of the network resources.

2) Comparisons: We compare FaiRTT with BBRv2 using above performance metrics in different experimental scenarios:

- Throughput with Time
- Fairness Index with Time
- Bottleneck link utilization with Time
- Throughput with Queue size
- Throughput with RTT

B. Analysis of Results

1) Throughput: In this experiment, we set a bottleneck bandwidth of 10 Mbps, a delay of 10 ms, and a bottleneck buffer size of 10 BDP. The RTT of the elephant and mice flows was set to 15ms and 5ms, respectively. From Fig. 3a, we can observe that regardless of the algorithms, elephant flows occupy a larger portion of throughput. BBRv2 presents highly suppressed mice flows by elephant flows with an average throughput ratio of 1.44 between the two competing flows in contrast to 1.05 in FaiRTT while maintaining the total throughput of the network. The average total throughput increased by 1% for FaiRTT as compared to BBRv2, respectively. Hence, FaiRTT is shown to be much-needed fairness when elephant and mice flows compete for the bottleneck bandwidth.

To evaluate the effect of bottleneck queue size on fairness, we configured the test topology having 10 Mbps bottleneck bandwidth and 10 ms bottleneck delay. The RTT of the elephant and mice flows was set to 15ms and 5ms, respectively. The bottleneck queue size was varied from 0.5 BDP to 100 BDP. Fig. 3b shows the throughputs of both elephant and mice flows for BBRv2 and FaiRTT with respect to the queue size. It is observed that fairness tends to decrease for larger queue sizes for BBRv2. The elephant flows occupy a much larger portion of the bottleneck bandwidth compared to mice flows particularly as the queue size increases from 10 BDP. For FaiRTT, both flows maintain almost stable and equitable throughput share for queue sizes more than 10 BDP. The algorithm performs best in terms of equitable throughput share at 2 BDP. For values less than 2 BDP, the performance of both algorithms is similar. The average throughput ratio between the competing flows is 1.46 for BBRv2 and 1.04 for FaiRTT.

We further evaluated the performance of FaiRTT for different RTTs. The RTT of elephant flows was varied from 5 to 30 ms and that of the mice flows was fixed to 5 ms. Fig. 3c displays throughput changes of both algorithms when mice flows compete with elephant flows with different RTTs for a 10 Mbps bottleneck bandwidth, 10 ms delay on a 10 BDP buffer size. It can be observed that as the RTT difference between the flows increases, the fairness between the flows deteriorates. The minimum and maximum throughput ratio between the competing flows 1.02 and 1.18 for FaiRTT considerably reduces as compared to 1.32 and 1.82 for BBRv2. The overall average throughput ratio is 1.11 for FaiRTT and 1.44 for BBRv2.

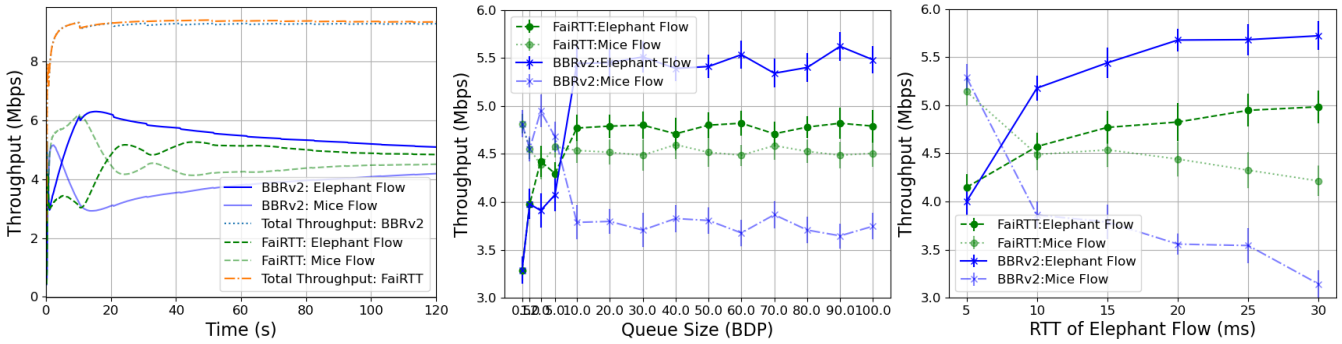
2) Jain's Fairness Index: Fig. 4a illustrates Jain's fairness index evaluation for both algorithms. We set bottleneck bandwidth at 10 Mbps and 10ms of delay. Elephant and mice flows have an RTT of 15 ms and 5 ms, respectively and they coexist on a 10 BDP bottleneck buffer size. For BBRv2, we recorded a minimum fairness index of about 0.88 and an average of 0.95 as compared to FaiRTT with 0.91 and 0.99. Fig. 4c shows the effect of varying bottleneck queue size from 0.5 BDP to 100 BDP on the fairness index for both algorithms. The fairness index exhibits the maximum value at 2 BDP for FaiRTT at 0.99 as compared to 0.95 for BBRv2. Both algorithms exhibit the same fairness for values less than 1 BDP. The average fairness index of FaiRTT and BBRv2 are 0.99 and 0.95, respectively. Fig. 4b illustrates the fairness index amongst competing elephant and mice flows with varying RTTs. Despite the fairness indexes' slight decrement when competing with elephant flows with increasing RTT, FaiRTT outperforms BBRv2. For competing flows of 5 ms and 30 ms, FaiRTT has a fairness index of 0.99 versus 0.93 for BBRv2. The maximum value of the fairness index obtained for FaiRTT is 0.99 as against 0.97 for BBRv2 at 5 ms and 10 ms competing flows. The average fairness index for FaiRTT and BBRv2 are 0.98 and 0.94, respectively. This demonstrates that FaiRTT enhances Jain's fairness index and can guarantee improved fairness amongst elephant and mice flows in different scenarios.

3) Bottleneck link utilization: Fig. 5 shows the bottleneck bandwidth utilization with the bandwidth set to 10 Mbps, delay at 10 ms, and a bottleneck buffer size of 10 BDP. FaiRTT achieves an average link utilization of 98.78% as compared to 97.21% in BBRv2. Hence, our proposed algorithm demonstrated its ability to optimize and utilize the available bottleneck bandwidth efficiently as compared to BBRv2.

V. Conclusion

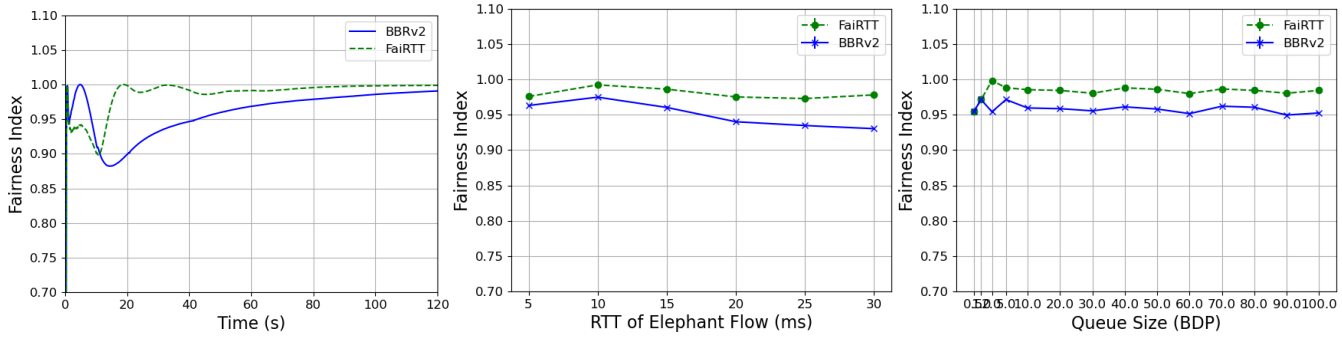
In this paper, we developed FaiRTT, a novel algorithm that dynamically estimates the BDP sending rate based on RTT measurements to achieve equitable bandwidth allocation between elephant and mice flows, addressing the drawback of BBR. In FaiRTT, we modeled the in-flight dependency on BDP, bottleneck bandwidth, and packet departure time after every `ACK`. We evaluated the performance of FaiRTT through extensive simulation experiments on multiple flows with different RTTs and buffer sizes using NS-3. The results show that FaiRTT has a better average throughput ratio of 1.08 between elephant and mice flows and an average fairness index of up to 0.98 compared to that of BBRv2 while improving the overall bottleneck link bandwidth utilization to 98.78%. By effectively resolving intra-protocol RTT fairness without compromising throughput on the bottleneck link, FaiRTT emerges as a promising solution to enhance overall network utilization and address the challenges posed by varying flow characteristics in next-generation networks. The findings of this study contribute valuable insights to the ongoing efforts in optimizing BBR for the evolving landscape of communication networks.

Authorized licensed use limited to: NATIONAL INSTITUTE OF TECHNOLOGY SURATHKAL. Downloaded on October 06,2026 at 11:13:40 UTC from IEEE Xplore. Restrictions apply.



(a) FaiRTT vs. BBRv2. (b) With different Queue Size Values. (c) With different RTT Values.

Fig. 3: Throughput Comparison.



(a) FaiRTT vs. BBRv2. (b) With different RTT Values. (c) With different Queue Size Values.

Fig. 4: Fairness Index.

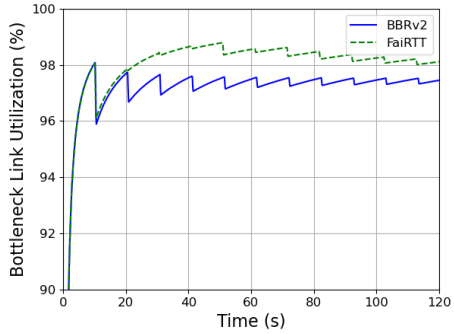


Fig. 5: Bottleneck Link Utilization of FaiRTT vs. BBRv2.

Acknowledgment and Disclaimer

This research is supported by the Ministry of Education, Singapore, under its Academic Research Tier 1 (Grant WBS number: R-R12-A405-0003).

Any opinions, findings, conclusions, or recommendations expressed in this material are those of the author(s) and do not reflect the views of the Ministry of Education, Singapore.

References

[1] N. Cardwell *et al.*, "BBR: Congestion-based Congestion Control," *ACM Queue*, vol. 14, no. 5, 2016.
[2] C. Chaccour *et al.*, "Seven defining features of terahertz (THz) wireless systems: A fellowship of communication and sensing," *IEEE Communications Surveys & Tutorials*, vol. 24, no. 2, pp. 967–993, 2022.
[3] L. Kleinrock, "Power and Deterministic Rules of Thumb for Probabilistic Problems in Computer Communications," in *IEEE ICC*, 1979.
[4] N. Cardwell *et al.*, "BBR Congestion Control," in *IETF 97th Meeting*, 2016.
[5] D. Scholz *et al.*, "Toward a Deeper Understanding of TCP BBR Congestion Control," in *IFIP Networking*, 2018.
[6] M. Hock *et al.*, "Experimental Evaluation of BBR Congestion Control," in *Proc. International Conference on Network Protocols (ICNP)*, 2017.
[7] S. Scherrer *et al.*, "Model-based Insights on the Performance, Fairness, and Stability of BBR," in *ACM IMC 2022*, 2022.
[8] C. K. Njogu *et al.*, "BBR-With Enhanced Fairness (BBR-EFRA): A New Enhanced RTT Fairness for BBR Congestion Control Algorithm," *Computer Communications*, vol. 200, pp. 95–103, 2023.
[9] W. Pan *et al.*, "Improvement of BBRv2 Congestion Control Algorithm Based on Flow-aware ECN," *Sec. and Commun. Netw.*, Jan. 2022.
[10] ——, "Improved RTT Fairness of BBR Congestion Control Algorithm Based on Adaptive Congestion Window," *Electronics*, vol. 10, no. 5, 2021.
[11] N. Cardwell *et al.*, "BBRv2: A Model-based Congestion Control," in *Proc. IETF 102th Meeting*, 2018.
[12] S. Zhang, "An evaluation of BBR and its variants," *arXiv preprint arXiv:1909.03673*, 2019.
[13] Y.-J. Song *et al.*, "Intra-protocol Convergence Problem in BBRv2's Bandwidth Probing," in *ICTC 2020*, 2020, pp. 1016–1018.
[14] A. Nandagiri *et al.*, "BBRv1 vs BBRv2: Examining Performance Differences through Experimental Evaluation," in *IEEE LANMAN 2020*, 2020.
[15] S. Ma *et al.*, "Fairness of congestion-based congestion control: Experimental evaluation and analysis," *arXiv preprint arXiv:1706.09115*, 2017.
[16] G.-H. Kim *et al.*, "Fairness Improvement of BBR Congestion Control Algorithm for Different RTT Flows," in *ICEIC 2019*, 2019.
[17] ——, "Enhanced BBR Congestion Control Algorithm for Improving RTT Fairness," in *ICUFN 2019*, 2019, pp. 358–360.
[18] G.-H. Kim and Y.-Z. Cho, "Delay-aware BBR Congestion Control Algorithm for RTT Fairness Improvement," *IEEE Access*, vol. 8, 2019.
[19] N. Cardwell *et al.*, "BBRv3: Algorithm Bug Fixes and Public Internet Deployment," in *Proc. IETF 117th Meeting*, 2023.
[20] J. A. Gomez Gaona, E. Kfoury, J. Crichigno, and G. Srivastava, "Evaluating TCP BBRv3 performance in wired broadband networks," 2023.
[21] R. Jain *et al.*, "A quantitative measure of fairness and discrimination," Digital Equipment Corporation, Hudson, MA, USA, Tech. Rep., 1984.

Authorized licensed use limited to: NATIONAL INSTITUTE OF TECHNOLOGY SURATHKAL. Downloaded on October 06,2026 at 11:13:40 UTC from IEEE Xplore. Restrictions apply.